

Robotics Assignments 1: Robot Kinematics

杨希渊
524531910015

April 26, 2026

1 Problem.1 Implementing Exponential Coordinates

All the functions in `kin_func_skeleton.py` are finished and tested correctedly. Code snippets are provided in attached files.

2 Problem 2. Forward Kinematics for 6 DOF Manipulators

The Stanford Arm is a 6-DOF manipulator consisting of five revolute joints and one prismatic joint (joint 3). The base frame $\{S\}$ is placed at joint 1 with the z -axis pointing upward. We set $l_0 = l_1 = 1$.

2.1 Initial Configuration $T_{ST}(0)$

When all joints are at their zero configuration, the end-effector coincides with the wrist center $q_w = (0, l_1, l_0) = (0, 1, 1)$:

$$T_{ST}(0) = g_{st0} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Unit Twists $\xi_1, \dots, \xi_6$

For each revolute joint, the twist is computed as $\xi_i = \begin{bmatrix} -\omega_i \times q_i \\ \omega_i \end{bmatrix}$, where ω_i is the unit rotation axis and q_i is any point on the axis. For a prismatic joint, $\xi_i = \begin{bmatrix} v_i \\ 0 \end{bmatrix}$, where v_i is the unit translation direction.

- **Joint 1** (vertical rotation, z -axis through origin): $\omega_1 = [0, 0, 1]^T$, $q_1 = [0, 0, 0]^T$, $v_1 = -\omega_1 \times q_1 = [0, 0, 0]^T$

$$\xi_1 = [0, 0, 0, 0, 1]^T$$

- **Joint 2** (shoulder pitch, $-x$ -axis at $(0, 0, l_0)$): $\omega_2 = [-1, 0, 0]^T$, $q_2 = [0, 0, l_0]^T$, $v_2 = -\omega_2 \times q_2 = [0, -l_0, 0]^T$

$$\xi_2 = [0, -1, 0, -1, 0, 0]^T$$

- **Joint 3** (prismatic, along $+y$): $v_3 = [0, 1, 0]^T$

$$\xi_3 = [0, 1, 0, 0, 0, 0]^T$$

- **Joint 4** (wrist, z -axis through q_w): $\omega_4 = [0, 0, 1]^T$, $v_4 = -\omega_4 \times q_w = [1, 0, 0]^T$

$$\xi_4 = [1, 0, 0, 0, 0, 1]^T$$

- **Joint 5** (wrist, $-x$ -axis through q_w): $\omega_5 = [-1, 0, 0]^T$, $v_5 = -\omega_5 \times q_w = [0, -1, 0]^T$

$$\xi_5 = [0, -1, 0, -1, 0, 0]^T$$

- **Joint 6** (wrist, y -axis through q_w): $\omega_6 = [0, 1, 0]^T$, $v_6 = -\omega_6 \times q_w = [-1, 0, 0]^T$

$$\xi_6 = [-1, 0, 0, 0, 1, 0]^T$$

2.3 Forward Kinematics Map

By the Product of Exponentials formula, the forward kinematics map is:

$$T_{ST}(\theta) = e^{[\xi_1]\theta_1} \cdot e^{[\xi_2]\theta_2} \cdot e^{[\xi_3]\theta_3} \cdot e^{[\xi_4]\theta_4} \cdot e^{[\xi_5]\theta_5} \cdot e^{[\xi_6]\theta_6} \cdot g_{st0}$$

where $[\xi_i]$ is the 4×4 matrix representation of twist ξ_i , θ_i is the joint displacement, and g_{st0} is the initial configuration from part (a).

3 Problem 3. Inverse Kinematics & Simulation (ARX X5)

3.1 Model Structure Analysis

Main XML File

The ARX X5 robot is defined by the URDF file `X5A.urdf`, which specifies all link geometries, joint parameters, and the kinematic tree structure.

Bodies and Kinematic Tree

The kinematic tree is a single serial chain from `base_link` to `link6`, with two gripper fingers and a camera mount branching from `link6`:

$$\text{base_link} \xrightarrow{j_1} \text{link1} \xrightarrow{j_2} \text{link2} \xrightarrow{j_3} \text{link3} \xrightarrow{j_4} \text{link4} \xrightarrow{j_5} \text{link5} \xrightarrow{j_6} \text{link6} \begin{cases} \xrightarrow{\text{fixed}} \text{camera} \\ \xrightarrow{j_7(\text{prismatic})} \text{link7 (finger 1)} \\ \xrightarrow{j_8(\text{prismatic})} \text{link8 (finger 2)} \end{cases}$$

All six arm joints are **revolute**. Joints 1–5 have limits of ± 10 rad (effectively unlimited), while joint 6 is bounded to $\pm \pi$. The gripper prismatic joints (7, 8) range from 0 to 0.044 m. Joint 8 is a mimic joint of joint 7 with ratio 1.0, so both fingers close symmetrically.

The joint axes are:

- Joint 1: z -axis Joint 2: y -axis Joint 3: y -axis
- Joint 4: y -axis Joint 5: z -axis Joint 6: x -axis

Geometries

Each link has both `<visual>` and `<collision>` geometry definitions using mesh files (STL format). The visual meshes include appearance materials (color, reflectance), while the collision meshes are used for physics-based collision detection. Both share the same STL geometry files, meaning no simplified collision hulls are provided.

Actuators

No `<transmission>` or `<actuator>` elements are defined in the URDF itself. Actuator parameters are specified externally in `config.yaml`: joint stiffness of 1000 and damping of 200 for both the arm and gripper. In our PyBullet implementation, position control mode (`POSITION_CONTROL`) is used with a maximum force of 500 N m, which effectively implements position-level actuation.

End-Effector Definition

The end-effector (tool frame) is defined as `link6`, as specified in `config.yml` via the `ee_link` parameter. The gripper fingers (`link7`, `link8`) and camera mount are attached to `link6` but are not part of the end-effector frame.

Kinematic Chain Construction

The URDF constructs the kinematic chain by nesting `<joint>` and `<link>` elements in parent-child order. Each revolute joint specifies its origin (position and orientation relative to the parent link), axis direction, and limits. The transformation from the base to any link is obtained by chaining the homogeneous transforms along the path from `base_link` to that link, forming the forward kinematics map.

3.2 Visualizations and Interactive Inverse Kinematics Control

The robot is loaded into PyBullet¹. Gravity is set to -9.8 m/s^2 along z , and the simulation runs at 240 Hz. Visualization results are demonstrated in Figure 1.

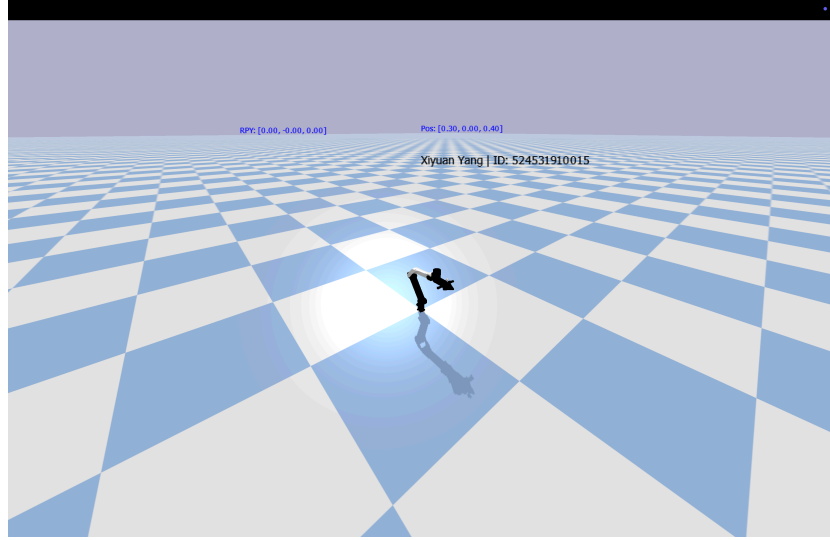


Figure 1: Visualization of ARX X5 robots in Pybullet

3.2.1 Mode 1: Keyboard Control

Keyboard inputs incrementally modify the target end-effector pose (position + = step, orientation + = step in Euler angles). At each simulation tick, the target pose is passed to PyBullet's `calculateInverseKinematics`, which solves for the joint angles that place the end-effector at the desired configuration. Table 1 shows detailed control mapping for different actions.

A detailed demo recording is provided in the attached video.

3.2.2 Mode 2: Target Pose Motion

In this mode, the robot is given an initial end-effector pose and a target pose, and inverse kinematics is used to compute the joint angle trajectory that drives the end-effector from the former to the latter. For demonstration purposes, the target position is **randomly** generated within a specified workspace region ($x \in [0.1, 0.5]$, $y \in [-0.3, 0.3]$, $z \in [0.2, 0.6]$), while the target orientation is kept the same as the current one.

The motion is executed over 240 interpolation steps (1 second at 240 Hz). At each step, the Cartesian-space pose is interpolated and then converted to joint angles via IK:

¹Using <https://piwheels.org/project/pybullet-mm/> for newer support on MacOS Systems.

Table 1: Keyboard control mappings for 6-DOF end-effector control.

Key	DOF	Action
i / k	x	Position $+/-$
j / l	y	Position $+/-$
u / o	z	Position $+/-$
1 / 2	Roll	Orientation $+/-$
3 / 4	Pitch	Orientation $+/-$
5 / 6	Yaw	Orientation $+/-$
0	—	Reset to initial pose

- **Position:** Linear interpolation (lerp): $\text{pos}(t) = \text{pos}_{\text{cur}} + t \cdot (\text{pos}_{\text{target}} - \text{pos}_{\text{cur}})$
- **Orientation:** Spherical linear interpolation (slerp) via `getQuaternionSlerp`, which avoids singularities in Euler-angle interpolation.

Each interpolated pose is fed to the IK solver, yielding a sequence of joint configurations that produce a smooth Cartesian-space trajectory. A detailed demo recording is provided in the attached video.

A Attached Files

- `kin_func_skeleton.py`: source code for problem 1.
- `problem_2.py`: source code for problem 2.
- `problem_3.py`: source code for problem 3.
- `demo1.mov`: Visualizations of keyboard control in Section 3.2.1. The robot can be controlled to different destinations with pressing different keyboards.
- `demo2.mov`: Visualizations of target pose control in Section 3.2.2.